

## SystemVerilog Assertion Practice Questions and Answers

**Write an assertion to check that once the signal `start` is asserted, the `done` signal goes high within exactly 4 clock cycles.**

```
property p_start_done_high;
    @(posedge clk) start |-> ##4 done;
endproperty
assert property (p_start_done_high);
```

**Write an assertion to verify that when `req` goes high, `ack` must go high within 2 to 5 clock cycles.**

```
property p_req_ack;
    @(posedge clk) req |-> ##[2:5] ack;
endproperty
assert property (p_req_ack);
```

**Write a concurrent assertion to check that when a `valid` signal is high, the `ready` signal should also be high within the same cycle.**

```
property p_valid_ready;
    @(posedge clk) valid |-> ready;
endproperty
assert property (p_valid_ready);
```

**Write an assertion that ensures after a `reset` signal goes low (de-asserted), a `data\_valid` signal must become high within 3 clock cycles and remain high for at least 2 cycles.**

```
property p_reset_data_valid;
    @(posedge clk) reset == 0 |-> ##[1:3] (data_valid == 1) ##1 (data_valid == 1);
endproperty

assert property (p_reset_data_valid);
```

**Write an assertion to check that if `write\_en` goes high, the `data\_out` signal must change within 2 clock cycles.**

```
property p_write_en_data_out;
    @(posedge clk) write_en |-> ##[1:2] (data_out != $past(data_out));
endproperty
assert property (p_write_en_data_out);
```

**Write an assertion to ensure that the signal `busy` is low when both `enable` and `reset` signals are high.**

```
property p_enable_reset_busy;
    @(posedge clk) (enable && reset) |-> !busy;
endproperty
assert property (p_enable_reset_busy);
```

**Write an assertion to check that the `valid` signal is not asserted for more than 3 consecutive clock cycles. (not asserted=not high)**

```
property p_valid_stability;
    @(posedge clk) valid |-> ##[1:3]!valid ;
endproperty
assert property (p_valid_stability);
```

**Write an assertion to verify that after a `read` operation is initiated (`read\_en` goes high), the `data\_out` signal should not change for 5 clock cycles unless `read\_en` goes low.**

```
property p_read_data_stable;
    @(posedge clk) (read_en) |-> (data_out == $past(data_out, 1) &&
        data_out == $past(data_out, 2) &&
        data_out == $past(data_out, 3) &&
        data_out == $past(data_out, 4) &&
        data_out == $past(data_out, 5));
endproperty
assert property (p_read_data_stable);
```

**Write an assertion to check that after the `load` signal is asserted, a `ready` signal must be asserted within 10 cycles, but not before 3 cycles.**

```
property p_load_ready;
    @(posedge clk) load |-> ##[3:10] ready;
endproperty
assert property (p_load_ready);
```

**Write an assertion that checks if `reset` is de-asserted, the `init\_complete` must go high exactly after 6 clock cycles and remain high for 2 cycles.**

```
property p_reset_init_complete;
    @(posedge clk) reset==0 |-> ##6 (init_complete==1) ##1 (init_complete==1);
endproperty
assert property (p_reset_init_complete);
```

**Write an assertion to check that a FIFO is never read (`read\_en` high) when it is empty (`fifo\_empty` high).**

```
property p_fifo_empty_read;
    @(posedge clk) (read_en && fifo_empty) |-> 0;
endproperty
assert property (p_fifo_empty_read);
```

**Write an assertion to verify that a burst transfer of 4 data cycles occurs (i.e., `data` must change 4 times in consecutive cycles).**

```
property p_burst_transfer;
    @(posedge clk) (start) |-> (data != $past(data)) ##1 (data != $past(data, 2)) ##1 (data != $past(data, 3)) ##1 (data != $past(data, 4));
endproperty
assert property (p_burst_transfer);
```

**Write an assertion to check that `write\_enable` is not asserted without a `valid\_address` being asserted.**

```
property p_write_valid_address;
    @(posedge clk) write_enable |-> valid_address;
endproperty
assert property (p_write_valid_address);
```

**Write an assertion to ensure if `reset` is de-asserted, the signal `counter` increments on every clock cycle for 5 consecutive clock cycles.**

```
property p_counter_increment;
    @(posedge clk) reset==0 |-> (counter == $past(counter) + 1) ##1 (counter == $past(counter, 2) + 1) ##1 (counter == $past(counter, 3) + 1) ##1 (counter == $past(counter, 4) + 1) ##1 (counter == $past(counter, 5) + 1);
endproperty
assert property (p_counter_increment);
```

**Write an assertion to check that `clk` toggles correctly (i.e., no glitches occur) for at least 100 clock cycles.**

```
property p_clk_toggling;
    @(posedge clk) clk |-> (clk != $past(clk));
endproperty
assert property (p_clk_toggling);
```

**Write an assertion to check that a sequence of events (`a`, `b`, `c`) happens in strict order, where each event occurs exactly 2 cycles apart.**

```
property p_event_order;
    @(posedge clk) (a) |-> ##2 (b) |-> ##2 (c);
```

```
endproperty  
assert property (p_event_order);
```

**Write an assertion to verify that two signals, `signal\_A` and `signal\_B`, never toggle in the same clock cycle.**

```
property p_signal_toggle;  
    @(posedge clk) !(signal_A && signal_B);  
endproperty  
assert property (p_signal_toggle);
```